Topics

Dynamic programming

- General method
- Matrix chain multiplication.
- Optimal Binary search trees.
- 0/1 Knapsack problem.
- All pairs shortest path problem
- Travelling sales person problem
- Reliability Design.

* Matrix chain multiplication: *

For the multiplication of 'n' matrices a series of operations are to be performed. A dynamic programming approach gives the optimal sequence or order of operations for this problem

→ Here we have to multiply 'n' matrices $A_1, A_2 .. A_n$, which can be accomplished by a series of matrix multiplication.

$$A_1 \times A_2 \times A_3 \times \cdots \times A_n.$$

→ We cannot change the above order of multiplication since matrix multiplication is associative but we can parenthesize the above multiplication.

For example.

consider three matrices

$A_{5\times4}$, $B_{4\times3}$. $C_{3\times5}$

multiplication cost of

$$[(AB)C] = (5\times4\times3) + (5\times3\times5) = 135$$

multiplication cost of

$$[A(BC)] = (4\times3\times5) + (5\times4\times5) = 160.$$

→ Therefore, we can see from the above illustrations that different evaluation sequences implies different cost of operation.

→ The problem is to determine the parenthesizing of the expression, defining, $A_1 \times A_2 \times A_3 \times \cdots \times A_n$ that ~~max~~ minimizes the total no. of scalar multiplications performed.

→ In the dynamic programming approach, we divide the problem into several sub-problems and then the solutions of these sub-problems are combined to get the solution of the original problem.

→ As we cannot change the order of multiplication, we parenthesize the expression to achieve optimum order of multiplying 'n' matrices.

→ Let A_{i-j} denote the multiplication of matrices through i to j and the order of A_{i-j} is $r_{i-1} \times r_j$.

We make a number of decisions, our job is to break the problem into several no. of sub-problems of same kind as the original problem and start parenthesizing the subsequences as the situation needs. Let us consider the highest level of parenthesizing where two matrix multiplications are considered i.e.

$$A_{1\ldots n} = A_{1\ldots k} \cdot A_{k+1\ldots n}$$

⇒ Now, the questions of interest are how can we split the order of matrices And how can we parenthesize the subsequences $A_{1\ldots k}$ and $A_{k+1\ldots n}$.

⇒ The answer to the first question is to try out all possible choices of k and pick out the best among them, but it does not lead you to the exponential growth.

⇒ The answer to the second question is to try out repeated by parenthesizing depending upon your need.

⇒ Another important thing to note about the matrix chain-multiplication problem is that, it is possible to characterize on optimal solution to a particular problem in terms of optimal solutions to its sub-problems. It does mean that "Principle of optimality" applies to this problem.

Dynamic programming approach computes the solutions of the subproblem and stores them into a table thereby avoiding the work of recomputing the solution whenever the sub-problem is encountered.

Let N_{ij} denotes the minimum no. of multiplications needed to compute $A_{i...j}$ where $1 \le i \le j \le n$.

⇒ If the sequence contains only one matrix i.e, if $i = j$ then the cost of operation is zero.

i.e, $N[i, j] = 0$.

⇒ If $i < j$ then the multiplication for $A_{i...j}$ is computed by considering each K.

$$A_{i...j} = A_{i...k}, A_{k+1...j} \text{ where } i \le k \le j.$$

⇒ $N[i, k]$ and $N[k+1, j]$ are the optimum multiplications needed to compute $A_{i...k}$ and $A_{k+1...j}$, since $r_{i-1} \times r_k$ is the order of $A_{i...k}$ and $r_k \times r_j$ is the order of $A_{k+1...j}$, the total no. of operations needed to multiply $A_{i...k}$ and $A_{k+1...j}$ is $r_{i-1} \cdot r_k \cdot r_j$.

∴ $N[i, j]$ can be computed as.

$$N[i, j] = \min N[i, k] + N[k+1, j] + r_{i-1} \cdot r_k \cdot r_j \text{ where } i < j \text{ and } i \le k < j.$$

Time complexity of the Algorithm:

⇒ The algorithm for the chain matrix multiplication using dynamic programming approach is given below.

```
Algorithm MATRIXMUL(A)
{
    begin
    integer i, j, K, b;
    for i ← 1 to n do
    {
    N[i, j] ← 0 // if i=j i.e, only one matrix.
    }
    for b ← 2 to n do
    {
        for j ← 1 to n-b+1 do
            j ← i+b-1
            N[i, j] ← ∞
        for k ← i to j-1 do
            N[i,j] ← min [N[i,K] + N[K+1, j] + γ_{i-1} · γ_K · γ_j]
        end
    end
end
end MATRIXMUL
}
```

=> In the above algorithm, there are three nested loops and each loop can iterate almost 'n' times which implies that above algorithm runs in $O(n^3)$ times.

Optimal Binary search Tree:

A binary search tree T is a binary tree, either it is empty or each node in the tree contains an identified and,

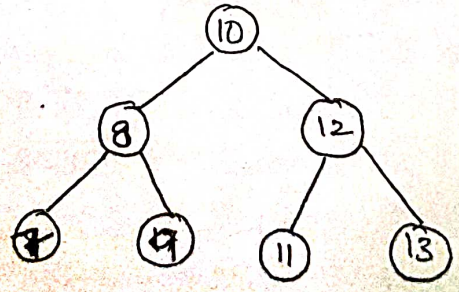
1) ALL identifiers in the left subtree of T are less Than (numerically or alphabetically) the identified in the root node T.
2) ALL identifiers in the right subtree are greater Than the identified in the root node T.
3) The left and right subtree of T are also binary search trees.

→ If We want to search an element in binary search tree, first That element is compared with root node. if element is less Than root node Then search continue in left subtree.

→ If element is greater Than root node Then search continue in right subtree.

→ If element is equal to root node Then print The successful search (element found) and terminate search procedure.

Example:



Example: The possible binary search trees for the identifier set.

$$(a_1, a_2, a_3) = (do, if, while) \quad \text{stop}$$

⇒ Here $n = 3$,

⇒ The possible no. of binary search trees are

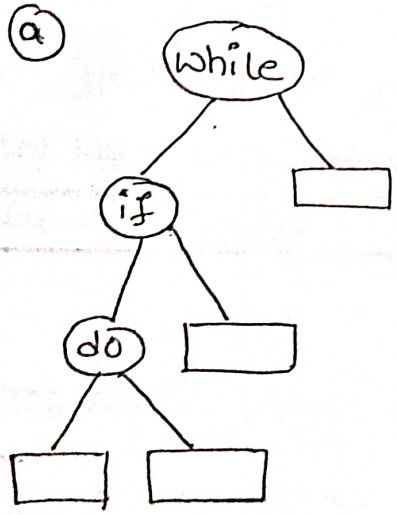
$$\frac{1}{n+1} 2nC_n$$

$$= \frac{1}{3+1} \cdot 2*3C_n \Rightarrow \frac{1}{4} \cdot 6C_n$$

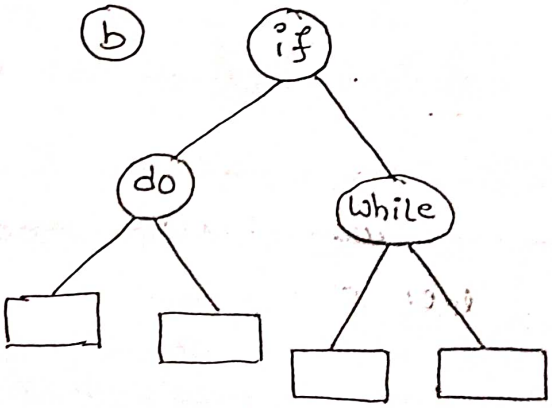
$$\Rightarrow \frac{1}{4} \times \frac{6 \times 5 \times 4 \times 3 \times 2 \times 1}{3 \times 2 \times 1 \times 3 \times 2 \times 1}$$

$$= 5$$

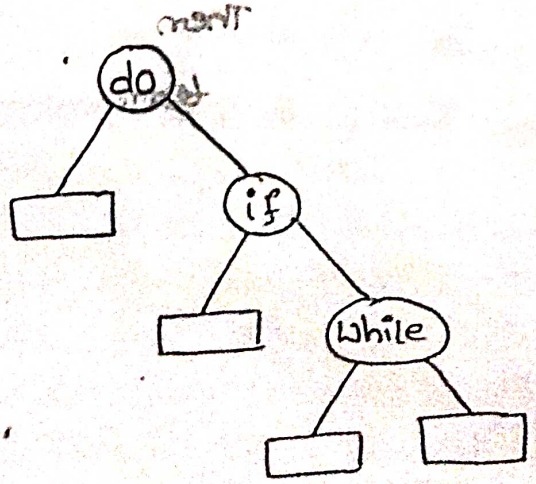
(a)



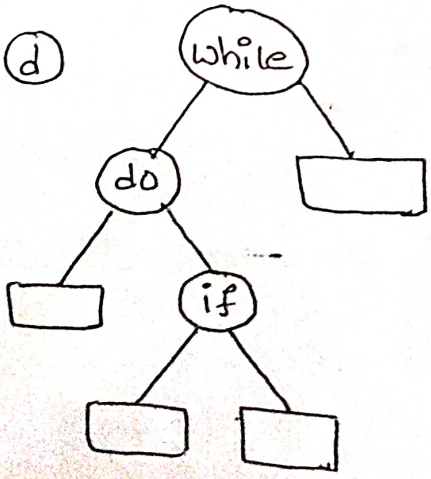
(b)



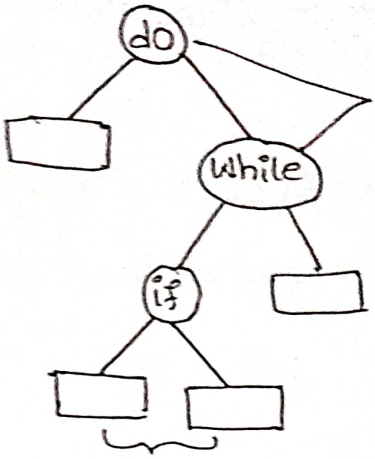
(c)



(d)



(e)



Internal nodes [successful search] [p_i]

External nodes [unsuccessful search (q_i)]

Cost of Binary search tree:

→ Let us assume That The given set of identifiers is $\{a_1, a_2, \ldots a_n\}$, with $a_1 < a_2 < a_3 < \ldots < a_n$.

→ Let $P(i)$ – Probability of searching an internal node i.e, internal probability.

$q(i)$ – Probability of searching an external node i.e, external probability.

→ The cost of binary search tree is

$$\sum_{1 \le i \le n} P(i) * level(a_i) + \sum_{0 \le i \le n} q(i) * (level(E_i) - 1)$$

→ cost of binary search for successful search.

$$\sum_{1 \le i \le n} P(i) * level(a_i)$$

→ Unsuccessful search:

$$\sum_{n < i < n} q(i) * level(E_i) - 1$$

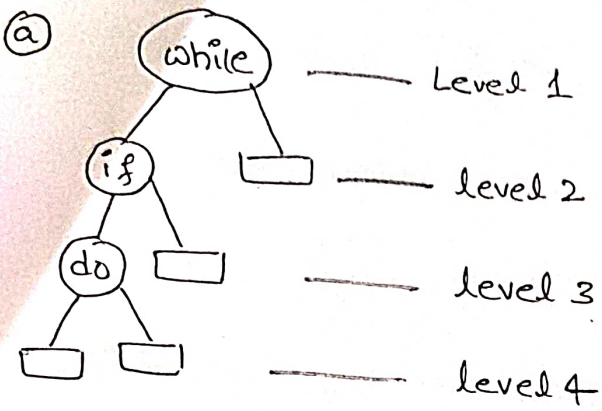
E → External node

⇒ **total search = successful search + Unsuccessful search**

Example: identifiers set $(a_1, a_2, a_3) = (do, if, while)$

Probabilities: $P(i) = q(i) = 1/7$ for all i.

(a)



Successful search: $\sum_{1 \le i \le n} P(i) * level(a_i)$

$= 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 3 \times \frac{1}{7}$

$= 6/7$

Unsuccessful search: $\sum_{0 \le i \le n} q(i) * (level(E_i) - 1)$

$= \frac{1}{7} * (2-1) + \frac{1}{7} * (3-1) + \frac{1}{7} * (4-1) + \frac{1}{7} * (4-1)$

$= \frac{1}{7} + \frac{2}{7} + \frac{3}{7} + \frac{3}{7}$

$= \frac{9}{7}$

∴ Total cost (T) $= \frac{6}{7} + \frac{9}{7}$

$= \frac{15}{7}$

(b) $1\times\frac{1}{7} + 2\times\frac{1}{7} + 2\times\frac{1}{7} + 2\times\frac{1}{7} + 2\times\frac{1}{7} + 2\times\frac{1}{7} + 2\times\frac{1}{7} = \frac{13}{7}$

(c) $1\times\frac{1}{7} + 2\times\frac{1}{7} + 3\times\frac{1}{7} + 1\times\frac{1}{7} + 2\times\frac{1}{7} + 6\times\frac{1}{7} = 15/7$

(d) $1\times\frac{1}{7} + 2\times\frac{1}{7} + 3\times\frac{1}{7} + 1\times\frac{1}{7} + 2\times\frac{1}{7} + 6\times\frac{1}{7} = 15/7$

(e) $1\times\frac{1}{7} + 2\times\frac{1}{7} + 3\times\frac{1}{7} + 1\times\frac{1}{7} + 2\times\frac{1}{7} + 3\times\frac{1}{7} + 3\times\frac{1}{7} = 15/7$

∴ In the above binary search tree, the cost of the tree 2 is minimum, hence it is optimal binary search tree.

Example:

Let $n=4$ and $(a_1, a_2, a_3, a_4) = (do, if, read, while)$

Let $p(1:4) = (3, 3, 1, 1)$ and

$q(0:4) = (2, 3, 1, 1, 1)$

Sol:

⇒ The number of possible binary search trees

$$= \frac{1}{n+2} {}^{2n}C_n = 14$$

$$W(i,j) = p(j) + q(j) + W(i, j-1) \quad \text{①}$$

$$c(i,j) = \min_{i<k\le j} \{ c(i, k-1) + c(k,j) \} + W(i,j) \quad \text{②}$$

$$r(i,j) = k$$

→ In order to solve above problem, first we will draw one table by taking 'i' corresponds to rows, 'j' corresponds to columns.

→ The cells in the table can be indicated as $W_{j,j+i}$, $C_{j,j+i}$, $R_{j,j+i}$.

$C(i,j)$ → It represent the cost of an optimal binary search tree t_{ij}

$r(i,j)$ → It represent the root of each tree t_{ij}.

Initially

$W(i,j) = q(i)$.

$C(i,j) = 0$

$r(i,j) = 0$.

$P(1:4) = (3,3,1,1)$

$q(0:4) = (2,3,1,1,1)$

	$j=0$	$j=1$	2	3	4
$i=0$	$W_{00}=2$ $C_{00}=0$ $r_{00}=0$	$W_{11}=3$ $C_{11}=0$ $r_{11}=0$	$W_{22}=1$ $C_{22}=0$ $r_{22}=0$	$W_{33}=1$ $C_{33}=0$ $r_{33}=0$	$W_{44}=1$ $C_{44}=0$ $r_{44}=0$
$i=1$	$W_{01}=8$ $C_{01}=8$ $r_{01}=1$	$W_{12}=7$ $C_{12}=7$ $r_{12}=2$	$W_{23}=3$ $C_{23}=3$ $r_{23}=3$	$W_{34}=3$ $C_{34}=3$ $r_{34}=4$	X
2	$W_{02}=12$ $C_{02}=19$ $r_{02}=1$	$W_{13}=9$ $C_{13}=12$ $r_{13}=2$	$W_{24}=5$ $C_{24}=8$ $r_{24}=3$	X	X
3	$W_{03}=14$ $C_{03}=25$ $r_{03}=2$	$W_{14}=11$ $C_{14}=19$ $r_{14}=2$	X	X	X
4	$W_{04}=16$ $C_{04}=32$ $r_{04}=2$	X			

$$W(i,j) = P(j) + q(j) + W(i,j-1)$$

$$c(i,j) = \min_{i<k\le j} \{ c(i,k-1) + c(k,j) \} + W(i,j)$$

Step 2:

(1) $W(0,1) = P(1) + q(1) + W(0,1-1)$

$= 3 + 3 + 2$

$= 8$

$c(0,1) = \min \{ c(0,1-1) + c(1,1) \} + W(0,1)$

k=1 $= \min \{ c(0,0) + c(1,1) \} + W(0,1)$

$= \min \{ 0 + 0 \} + 8$

$= 8$

$r_{01} = k \Rightarrow r_{01} = 1$

(2) $W(1,2) = P(2) + q(2) + W(1,1)$

$= 3 + 1 + 3 = 7$

$c(1,2) = \min \{ c(1,1) + c(2,2) \} + W(1,2)$

k=2 $= 0 + 0 + 7 = 7$

$r_{12} = k \Rightarrow r(1,2) = 2$

(3) $W(2,3) = P(3) + q(3) + W(2,2)$

$= 1 + 1 + 1 = 3$

$c(2,3) = \min \{ c(2,2) + c(3,3) \} + W(2,3)$

k=3 $= 0 + 0 + 3 = 3$

$r(2,3) = k \Rightarrow r(2,3) = 3$

(4) $W(3,4) = P(4) + q(4) + W(3,3)$

$= 1 + 1 + 1 = 3$

$c(3,4) = \min \{ c(3,3) + c(4,4) \} + W(3,4)$

Step 3:

① $w(0,2) = P(2) + q(2) + w(0,1)$

$= 3+1+8 = 12$

$c(0,2) = \min_{k=1,2} \left\{ \begin{matrix} c(0,0) + c(1,2), \\ c(0,1) + c(2,2). \end{matrix} \right\} + w(i,j)$

$= \min \left\{ \begin{matrix} 0+7 \\ 8+0 \end{matrix} \right\} + 12$

$c(0,2) = 7+12 = 19$

$[\because$ $k=1$ gives the minimum value $r(0,2) = 1]$

$r(0,2) = k \Rightarrow r(0,2) = 1$

② $w(1,3) = P(3) + q(3) + w(1,2)$

$= 1+1+7 = 9$

$c(1,3) = \min_{k=2,3} \left\{ \begin{matrix} c(1,1) + c(2,3) \\ c(1,2) + c(3,3) \end{matrix} \right\} + w(i,j)$

$= \min \left\{ \begin{matrix} 0+3 \\ 7+0 \end{matrix} \right\} + 9$

$= 3+9 = 12$

$r(1,3) = 2$

③ $w(2,4) = P(4) + q(4) + w(2,3)$

$= 1+1+3 = 5$

$c(2,4) = \min_{k=3,4} \left\{ \begin{matrix} c(2,2) + c(3,4) \\ c(2,3) + c(4,4) \end{matrix} \right\} + w(2,4)$

$= \min \left[\begin{matrix} 0+3, \\ 3+0 \end{matrix} \right] + 5$

$= 3+5 = 8$

$r(2,4) = 3$ (or) 4.

Step4:

① $W(0,3) = P(3) + Q(3) + W(0,2)$

$= 1+1+12 = 14$

$C(0,3) = \min_{K=1,2,3} \{ C(0,0)+C(1,3),\ C(0,1)+C(2,3),\ C(0,2)+C(3,3) \} + W(0,3)$

$= \min \{ 0+12,\ 8+3,\ 19+0 \} + 14$

$= 11+14 = 25$

$r(0,3) = 2$

② $W(1,4) = P(4) + Q(4) + W(1,3)$

$= 1+1+9 = 11$

$C(1,4) = \min_{K=2,3,4} \{ C(1,1)+C(2,4),\ C(1,2)+C(3,4),\ C(1,3)+C(4,4) \} + W(1,4)$

$= \min \{ 0+8,\ 7+3,\ 12+0 \} + 11$

$= 8+11 = 19$

$r(1,4) = 2$

Step 5:

$W(0,4) = P(4) + Q(4) + W(0,3)$

$= 1+1+14 = 16$

$C(0,4) = \min_{K=1,2,3,4} \{ C(0,0)+C(1,4),\ C(0,1)+C(2,4),\ C(0,2)+C(3,4),\ C(0,3)+C(4,4) \} + W(0,4)$

$C(0,0)+C(1,4) = 0+19 = 19$

$C(0,1)+C(2,4) = 8+8 = 16$

$C(0,2)+C(3,4) = 19+3 = 21$

$C(0,3)+C(4,4) = 25+0 = 25$

$= 16+16$

$= 32$

$C(0,4) = 32$

$r(0,4) = k = 2$

→ Now observe the table, the last cell i.e, 4th row and 4th column, contains

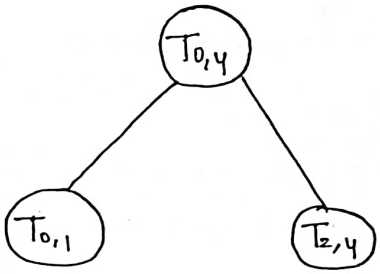
$r_{04} = 2$, i.e, $r_{04} = a_2$ (2 corresponds to second node a_2).

$r_{04} = k$, Then $k = 2$

→ Let T be the optimal binary search tree.

$$T_{i,j} = T_{i,k-1}, T_{k,j}$$

$T_{0,4}$ is divided into two parts $T_{0,1}$ and $T_{2,4}$ (since $k=2$)



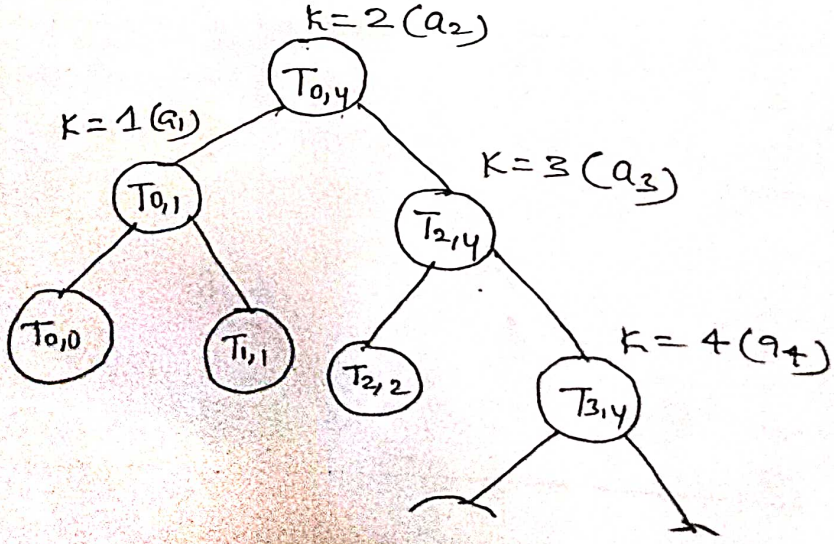
→ Now observe $T_{0,1}$, $T_{2,4}$ i.e, $R_{0,1}$, $R_{2,4}$

$R_{0,1} = 1$, i.e, $k=1$, $R_{2,4} = 3$ i.e, $k=3$

→ $T_{0,1}$ is divided into $T_{0,0}$ and $T_{1,1}$ $(k=1)$,

→ $T_{2,4}$ is divided into $T_{2,2}$ and $T_{3,4}$ $(k=3)$

→ Again $T_{3,4}$ is divided into $T_{3,3}$ and $T_{4,4}$ $(k=4)$.



Maximize $\prod_{1 \le i \le n} \phi_i(m_i)$

Subject to $\sum_{1 \le i \le n} c_i m_i \le c$

$m_i \ge 1$ and integer, $1 \le i \le n$

→ We may assume that each $c_i > 0$ each m_i must be in The range $1 \le m_i \le u_i$ can be calculated as follows.

$$u_i = \left[\left(c + c_i - \sum_{j=1}^{n} c_j \right) / c_i \right]$$

Reliability Design Problem:

We are to design a Three stage system with device types D_1, D_2, D_3. The costs are \$30, \$15, \$20 respectively. The cost of The system is to be no more Than \$105.

→ The reliability of each device type is 0.9, 0.8, 0.5

$c_1 = 30$, $c_2 = 15$, $c_3 = 20$, $c = 105$,

$r_1 = 0.9$, $r_2 = 0.8$, $r_3 = 0.5$

→ u_i = Upper bound $= \left[\left(c + c_i - \sum_{1}^{n} c_j \right) / c_i \right]$

$u_1 = [(105 + 30 - (30 + 15 + 20)) / 30]$

$= [(135 - 65) / 30]$

$= 2.333$

$= 2$

$u_2 = [(105 + 15 - (30 + 15 + 20)) / 15]$

$$U_3 = \left[(105 + 20 - (30 + 15 + 20)) / 20 \right]$$

$$= [(125 - 65) / 20]$$

$$= 3$$

$\because U_1 = 2, \quad U_2 = 3, \quad U_3 = 3.$

→ $Q_i(m_i)$ is a reliability function.

$$\boxed{Q_i(m_i) = 1 - (1 - r_i)^{m_i}}$$

→ Each pair in this is represented by (r, x) where as r is reliability, x is a cost.

→ Using S_j^i to represent all tuples of obtainable from s^{i-1} by choosing $m_i = j$.

→ Begining with $\boxed{S^0 = \{(1,0)\}}$

→ Since $1 \leq m_i \leq U_i$ i.e, $1 \leq m_1 \leq 2$,

$m_1 = 1$ or 2, it means $j = 1$ or 2.

$i = 1$, $j = 1$, i.e $m_1 = 1$

$$Q_1(m_1) = 1 - (1 - r_1)^1 = 1 - (1 - 0.9) = 1 - 0.1 = 0.9$$

$$S^0 = \{(1,0)\}$$

→ In each type (r, x) reliability is multiplied with previous reliability and cost is added to previous cost.

$$S_1^1 = \{(0.9), 30)\}$$

→ Here reliability (0.9) is multiplied with 1 and cost $c_1 = 30$ is added to 0. It means the operation is

$i=1$, $j=2$, i.e $m_1=2$

$\phi_1(m_1) = 1-(1-r_1)^2 = 1-(1-0.9)^2 = 1-0.01 = 0.99$

→ Now, 0.99 is multiplied with 1 and cost $2c_1 = 60$ (since j

added to 0.

$$S_2' = \{(0.99, 60)\}$$

→ S^1 can be obtained by merging the sets S_1', S_2'

$$\boxed{S^1 = \{(0.9, 30), (0.99, 60)\}}$$

→ We can't calculate S_1^3 since upper bound is 2 i.e, $U_1 = 2$.

∴ Similarly, S_1^2, S_2^2, S_3^2 can be calculated from S^1.

S_1^2, here $j=1$ i.e $m_2=1$, $i=2$.

$$\phi_2(m_2) = 1-(1-r_2)^{m_2} = 1-(1-0.8)^1 = 1-0.2 = 0.8$$

$$S_1^2 = \{(0.9\times0.8, 30+15), (0.99\times0.8, 60+15)\}$$

$$= \{(0.72, 45), (0.792, 75)\}$$

S_2^2, $j=2$, $i=2$, $m_2=2$

$$\phi_2(m_2) = 1-(1-r_2)^2 = 1-(1-0.8)^2 = 1-(0.2)^2 = 0.96$$

→ since $j=2$, $2c_2 = 2\times15 = 30$ is added to S^1 pairs.

$$S_2^2 = \{(0.9\times0.96, 30+30), (0.99\times0.96, 60+30)\}$$

$$= \{(0.8649, 60), (0.9504, 90)\}$$

$$\therefore S_2^2 = \{(0.8649, 60), (0.9504, 90)\}$$

S_3^2, $i=2$, $j=3$, $m_2=3$

$\phi_2(m_2) = 1-(1-r_2)^{m_2} = 1-(1-0.8)^3 = 1-(0.2)^3 = 0.992$

since $j=3$, $3C_2 = 3\times 15 = 45$

$S_3^2 = \{(0.9\times 0.992, 30+45), (0.99\times 0.992, 60+45)\}$

$= \{(0.8928, 75)\ (0.98208, 105)\}$

$S^2 = S_1^2 \cup S_2^2 \cup S_3^2$

$= \{(0.72, 45)\ (0.792, 75), (0.864, 60), (0.9504, 90),$
$(0.8928, 75), (0.98208, 105)\}$

→ Now apply ~~Purging~~ Purge rule, Then (0.792, 75) pair is removed from S^2.

$S^2 = \{(0.72, 45), (0.864, 60), (0.9504, 90), (0.8928, 75), (0.98208, 105)\}$

⟹ Observe pairs, cost is in increasing order.

i.e $45<60<90<75$ here $90<75$, condition fails

→ Pair (0.90504, 90) is not in an appropriate order so remove (0.9504, 90) from S^2.

$S_2 = \{(0.72, 45), (0.864, 60), (0.8928, 75), (0.98208, 105)\}$

→ similarly S_1^3, S_2^3, S_3^3 can be calculated from S^2

S_1^3, $i=3$, $j=1$, so $m=1$,

$\phi_3(m_3) = 1-(1-r_3)^{m_3} = 1-(1-0.5)^1 = 0.5$

$S_1^3 = \{(0.72 \times 0.5, 45+20), (0.864 \times 0.5, 60+20), (0.8928 \times 0.5, \ldots$

$(0.98208 \times 0.5, 105+20)\}$

→ last tuple is removed from S_1^3, since cost 125 exceeding the given cost 105.

$S_1^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$

S_2^3, $i=3$, $j=2$, $m_3=2$

→ since $j=2$, $2C_3 = 2 \times 20 = 40$

$\phi_3(m_3) = 1-(1-r_3)^{m_3} = 1-(1-0.5)^2 = 1-0.25 = 0.75$

$S_2^3 = \{(0.72 \times 0.75, 45+40)(0.864 \times 0.75, 60+40)\}$

$= \{(0.54, 85), (0.648, 100)\}$

S_3^3, $i=3$, $j=3$, $m_3=3$

→ since $j=3$, $3C_3 = 3 \times 20 = 60$

$\phi(m_3) = 1-(1-r_3)^{m_3} = 1-(1-0.5)^3 = 1-0.125 = 0.875$

$S_3^3 = \{(0.72 \times 0.875, 45+60)\}$

$= \{(0.63, 105)\}$

$S^3 = S_1^3 \cup S_2^3 \cup S_3^3$

$= \{(0.36,65), (0.432,80), (0.4464,95), (0.54,85), (0.648,100), (0.63,105)\}$

→ Apply Purge and Dominance Rule

$S^3 = \{(0.36,65)\ (0.432,80)\ (0.54,85)\ (0.648,100)\}$

→ The best design has reliability of 0.648 and a cost of 100, (0.648, 100) pair present in S_2^3, i=3 and j=2, so $m_3 = 2$, The (0.648, 100) pair obtained from (0.864, 60), which is present in S_2^2, i=2 and j=2, So, $m_2 = 2$. (0.864, 60) pair obtained from (0.9, 30), which is in S_1^1, i=1 and j=1, So $m_1 = 1$.

$\therefore m_1 = 1$, $m_2 = 2$, $m_2 = 2$

Travelling Salesperson Problem

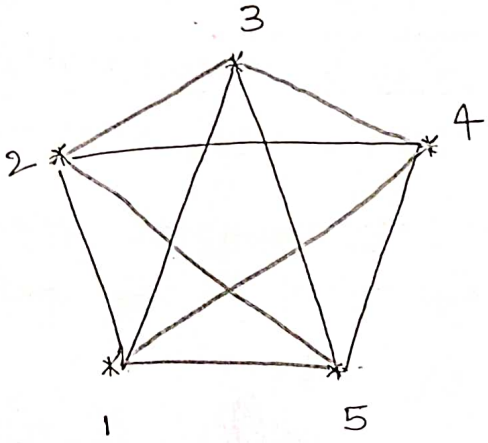
Here the salesman should start at a point and travels all the places and comes back to starting point. The problem is to minimize the traveling cost.

Applications of TSP:

① A postal van is to pick up mail from mail boxes located at n different sites. The route taken by a postal van is a tour and the problem is to find a tour of minimum length.

② A minimum cost tour of a robot arm used to tighten the nuts on some piece of machinery on an assembly line will minimize the time needed for the arm to complete its task.

Example:



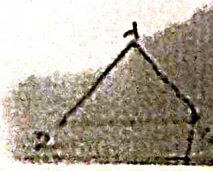
→ The feasible solution could be.

1-5-2-3-4-1 is same as

2-3-4-1-5-2

→ So there are n! feasible solutions

Ex:



it is not hamiltonian graph

→ Assume a tour to be a simple path that starts and ends at vertex 1.

→ Let $G = (V, E)$ be directed graph with edge costs c_{ij}, $c_{ij} > 0$ for all i and j and $c_{ij} = +\infty$ if $\langle i, j \rangle \notin E$.

→ Let $|V| = n$ and assume $n > 1$.

→ A tour of G is a directed cycle that includes every vertex in V.

→ The cost of a tour is the sum of the cost of the edges on the tour.

→ The TSP problem is to find a tour of minimum cost.

→ Assume a tour to be a simple path that starts and ends at vertex 1.

→ Every tour consists of an edge $\langle 1, k \rangle$ for some $k \in V - \{1\}$ and a path from vertex k to vertex 1 goes through each vertex in $V - \{1, k\}$, exactly once.

→ If the tour is optimal, then the path from k to 1 must be a shortest, k to 1 path going through all vertices in $V - \{1, k\}$

→ Hence the principle of optimality holds.

→ Let $g(i, S)$ be the length of a shortest path starting at vertex I, going through all vertex in S and terminating at vertex 1.

→ $g(1, V - \{1\})$ is the length of an optimal salesperson tour.

From The Principle of optimality it follows that

$$g(1, V-\{1\}) = \min_{2 \le k \le n} \{C_{1k} + g(k, V-\{1,k\})\}$$

→ In general, for $i \notin S$

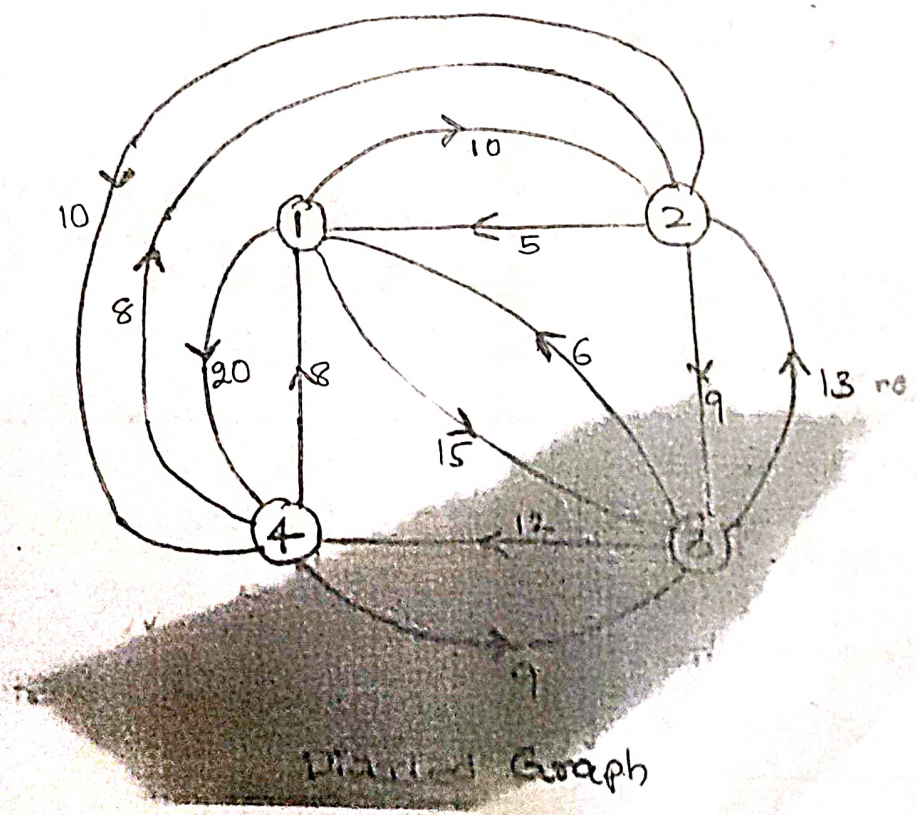
$$g(i,S) = \min_{j \in S} \{C_{ij} + g(j, S-\{j\})\} \quad \rightarrow (1)$$

$$g(i,\emptyset) = C_{i1}, \quad 1 \le i \le n$$

→ $g(i,\emptyset)$ is The length of The shortest path Starting at i going Through no vertex and ending at 1.

→ Hence, we can use (1) to obtain $g(i,S)$ for all S of size 1. Then we can obtain $g(i,S)$ for S with $|S| = 2$, and so on.

Example:



Directed Graph

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

⇒ Edge length matrix

$\Rightarrow g(i,\phi) = C_{i,1}, \quad 1 \le i \le n$

$g(1,\phi) = C_{11} = 0$

$g(2,\phi) = C_{21} = 5$

$g(3,\phi) = C_{31} = 6$

$g(4,\phi) = C_{41} = 8$

$\Rightarrow g(i,s) = \min \{ C_{ij} + g(j, s-\{j\}) \}$

$|s| = 1$

It means That set contains only one element. Before solving This problem, we make an assumption That The salesperson stasts at vertex 1, from That he can move to vertex 2. if he visits vertex 2, from That vertex, he can next visit either vertex 3 or Vertex 4. since $|s|=1$.

(i) $g(2,3)$

(ii) $g(2,4)$

→ From vertex 1, next he can visit Vertex 3 instead of vertex 2, in This case from vertex 3, next he can visit either 2 or 4.

(iii) $g(3,2)$

(iv) $g(3,4)$

→ From vertex 1, he can visit either vertex 4 instead of vertex 3, in This case from vertex 4, next he can visit either 2 or 3.

(v) $g(4,2)$

(vi) $g(4,3)$

All These 6 possibilities can be determined using (1) as follows.

$$g(2,3) = \min_{j \in S} \{C_{23} + g(3,\emptyset)\}$$

$$= 9+6$$

$$= 15$$

$$g(2,4) = \min_{j \in S} \{C_{24} + g(4,\emptyset)\}$$

$$= 10+8$$

$$= 18$$

$$g(3,2) = \min \{C_{32} + g(2,\emptyset)\}$$

$$= 13+5$$

$$= 18$$

$$g(3,4) = \min \{C_{34} + g(4,\emptyset)\}$$

$$= 12+8$$

$$= 20$$

$$g(4,2) = \min \{C_{42} + g(2,\emptyset)\}$$

$$= 8+5$$

$$= 13$$

$$g(4,3) = \min \{C_{43} + g(3,\emptyset)\}$$

$$= 9+6$$

$$= 15$$

$|S| = 2$

Here set contains two values, so we can place two vertices in S. From starting vertex 1, next he can visit either vertex 2 or 3 or 4.

→ if he visits vertex 2, Then from That vertex he can visit either 3 or 4; so determine $g(2, \{3,4\})$, ($s=\{3,4\}$ since $|s|=2$), $g(3, \{2,4\})$ and $g(4, \{2,3\})$.

$$g(2, \{3,4\}) = \min_{j \in s} \{C_{23} + g(3,4),\ C_{24} + g(4,3)\}$$

$$= \min_{j \in s} \{9+20,\ 10+15\}$$

$$= 25$$

$$g(3, \{2,4\}) = \min_{j \in s} \{C_{32} + g(2,4),\ C_{34} + g(4,2)\}$$

$$= \min \{13+18,\ 12+13\}$$

$$= 25$$

$$g(4, \{2,3\}) = \min \{C_{42} + g(2,4),\ C_{43} + g(3,2)\}$$

$$= \min \{8+15,\ 9+18\}$$

$$= 23$$

$|s|=3$,

Here s contains 3 elements, After starting from vertex 1, next he can visit 2 or 3 or 4 vertices. so, determine $g(1, \{2,3,4\})$.

$$g(1, \{2,3,4\}) = \min_{j \in s} \{C_{12} + g(2, \{3,4\}),\ C_{13} + g(3, \{2,4\}),\ C_{14} + g(4, \{2,3\})\}$$

$$= \min \{10+25,\ 15+25,\ 20+23\}$$

$$= \min \{35, 40, 43\}$$

$$= 35$$

0/1 Knapsack problem

In olden days, there was a store, assume that, which contains different types of ornaments, which are made up of gold. Let n_1, n_2, n_3 be ornaments, cost and weight of these ornaments are c_1, c_2 and c_3 dollars, w_1, w_2, w_3 pounds respectively.

→ Now a thief wants to rob the ornaments such that he should get maximum profit. In this, the thief can't place fraction of ornament in the bag i.e., either he can place ornament completely in the bag or he can't place ornament so,

$x_i = 0$ or 1

$x_i = 0$ means we cannot place ornament in the bag.

$x_i = 1$ means we can place ornament completely in the bag.

→ This problem contains either 0 or 1, thats why this problem is called as 0/1 knapsack problem. In this we can't place fractional weights in the knapsack.

→ Here, either no item is to be put or the complete item should be in the bag. Fraction of item can't be placed in the bag. Consider weights $w_1, w_2, \ldots w_n$ and fraction of weights to be put in the bag should be $x_1, x_2, \ldots x_n$, $x_i = 0$ or 1. The dynamic programming solution for 0/1 knapsack is.

$$F_n(M) = \text{Max}\left[F_{n-1}(M),\ F_{n-1}(M-W_n) + P_n\right]$$

↓ when, $x_n = 1$ ↓ when $x_n = 0$

M = size of knapsack.

→ When $x_n = 1$ then the size of the bag is reduced by w_n which is the weight of the n^{th} item. As we are placing the n^{th} item we should add the profit for the last item. Similarly, we find for $F_{n-1}(M)$ and so on up to $F_1(M)$.

$$S_1^i = \left\{ (P_i, W_i) \,/\, (P - P_i,\ W - W_i) \in S^{i-1} \right\}$$

→ if we try to remove the i^{th} item, then the profit of i^{th} item reduced from the total profit and weight of i^{th} item is removed from the total weight which belongs to the profit and weight of $(i-1)^{th}$ item.

$$S_0^0 = \{0, 0\}$$

$$S_1^i = S^{i-1} + (P_i, W_i) \quad \text{addition}$$

$$S^i = S^{i-1} + S_1^i \quad \text{merging operation}$$

Consider, $n = 3$, $M = 6$

$(W_1, W_2, W_3) = (2, 3, 4)$.

$(P_1, P_2, P_3) = (1, 2, 5)$

$S_0^0 = \{0,0\}$

$S_1^1 = S^{i-1} + (P_i, W_i)$

$S_1^1 = S^0 + (P_1, W_1)$

$= \{(0,0)\} + \{(1,2)\}$

$= \{(1,2)\}$

$S^1 = S^0 + S_1^1$

$= \{(0,0)\} + \{(1,2)\}$

$= \{(0,0), (1,2)\}$

$S_1^2 = S^1 + (P_2, W_2)$

$= \{(0,0)(1,2)\} + (2,3)$

$= \{(0,0)(1,2)\} + \{(2,3)\}$

$= \{(2,3)(3,5)\}$

$S^2 = S^1 + S_1^2$

$= \{(0,0)(1,2)\} + \{(2,3)(3,5)\}$

$= \{(0,0)(1,2)(2,3)(3,5)\}$

$S_1^3 = S^2 + (P_3, W_3)$

$= \{(0,0)(1,2)(2,3)(3,5)\} + (5,4)$

$= \{(5,4)(6,6)(7,7)(8,9)\}$

$S^3 = S^2 + S_1^3$

$= \{(0,0)(1,2)(2,3)(3,5)\} + \{(5,4)(6,6)(7,7)(8,9)\}$

$= \{(0,0)(1,2)(2,3)(3,5)(5,4)(6,6)(7,7)(8,9)\}$

Purging Rule (Dominance Rule): If one of S^{i-1} and S^i_1 has a pair (P_j, W_j) and other has a pair (P_k, W_k) and $P_j \leq P_k$ while $W_j \geq W_k$ then the pair (P_j, W_j) is discarded.

has to be discarded bcoz 3<5, 5>4 (3,5) (5,4)

$S^3 = \{(0,0)\ (1,2)\ (2,3)\ (3,5)\ (5,4)\ (6,6)\ (7,7)\ (8,9)\}$

S^2 S^3

→ After applying Purge rule, we will check the following condition inorder to find solution.

If $(P_i, W_i) \in S^n$ and $(P_i, W_i) \notin S^{n-1}$ then

$X_n = 1$

$X_3 = 1$

Otherwise $X_n = 0$

$(6,6) \in S^3$

$(6,6) \notin S^2$

$X_3 = 1$

$(6-P_3,\ 6-W_3) = (6,6) - (5,4) = (1,2) \in S^2$

$(1,2) \notin S^1 \rightarrow$ False

$X_2 = 0$

$(1,2) \in S^1$

$(1,2) \notin S^0 \rightarrow$ True

$X_1 = 1$

$\therefore X_1 = 1,\ X_2 = 0,\ X_3 = 1$

Maximum profit is,

$$\sum P_i X_i = P_1X_1 + P_2X_2 + P_3X_3$$
$$= 1\times1 + 2\times0 + 5\times1$$
$$= 1 + 5$$
$$= 6$$

Here, $(P_i, W_i) = (6,6)$ in the above problem so $(P_i, W_i) \notin S^2$, condition becomes true. So $x_3 = 1$. i.e, 3rd item is placed entirely in bag. To get next item to be placed in bag subtract (P_3, W_3) from $(6,6)$ because W_3 is placed in bag. If $(1,2)$ belong to $S^{2-1} = S^1$. Here condition becomes false, so $x_2 = 0$. No quantity of second item can be placed in bag. Inorder to get next item to be placed in bag. check $(P_1, W_1) \in (1,2)$ $(1,2) \notin S^0$. here condition becomes true so $x_1 = 1$.

Optimal Solution is, $(x_1, x_2, x_3) = (1,0,1)$

Max profit is $\sum P_i x_i = 6$.

Time complexity of the Algorithm:

```
Algorithm DKP (P, W, n, M)
{
    S^0 ← {(0,0)}
    for i ← 1 to n-1 do
        S_1^i ← {(P_i, W_i) (P-P_i, W-W_i) ∈ S^{i-1} and W_i ≤ M}
        S^i ← MERGE_PURGE (S^{i-1}, S_1^i)
    repeat
    (P_x, W_x) ← last tuple in S^{n-1}
(P_y, W_y) ← (P_i + P_n, W_i + W_n) where W_i is the largest w in any
                                    tuple in S^{n-1} such that w+W_n ≤ M.
    if P_x > P_y ) then x_n ← 0
    else x_n ← 1
        end
    end if
    end DKP
```

* Time complexity of knapsack problem is $O(n)$.

* Time complexity of knapsack problem is $O(2^{n/2})$.

Ex: 2 , N=4, W=5.

(P, W)
(7, 5)

$(w_1, w_2, w_3, w_4) = (2, 3, 4, 5)$

$(P_1, P_2, P_3, P_4) = (3, 4, 5, 6)$

$S^0 = \{0, 0\}$

$S_1^1 = S^0 + (P_1, W_1) = \{0,0\} + (3,2) = \{(3,2)\}$

$S^1 = S^0 + S_1^1 = \{(0,0), (3,2)\}$

$S_1^2 = S^1 + (P_2, W_2) = \{(0,0), (3,2)\} + (4,3)$

$= \{(4,3), (7,5)\}$

$S^2 = S^1 + S_1^2 = \{(0,0)\ (3,2)\ (4,3)\ (7,5)\}$

$S_1^3 = S^2 + \{P_3, W_3\} = \{(5,4)(8,6)(9,7)(12,9)\}$

$[5,4)$

$S^3 = S^2 + S_1^3 = \{(0,0)(3,2)(4,3)(7,5)(5,4)(8,6)(9,7)(12,9)\}$

$S_1^4 = S^3 + \{P_4, W_4\} = \{(6,5)\ (9,7)(10,8)(13,10)(14,9)(14,11)(15,12)(18,14)\}$

$[6,5)$

$S^4 = S^3 + S_1^4 = \{(0,0)\ (3,2)\ (4,3)\ (7,5)\ (5,4)\ (8,6)\ (9,7)\ (12,9), (6,5), (9,7)$

$(10,8)\ (13,10)\ (11,9)\ (14,11)\ (15,12)\ (18,14)\}$

$= \{(0,0)(3,2)(4,3)(7,5)(5,4)(6,5)\}$

$(7,5) \in S^4, \in S^3 \Rightarrow x_4 = 0$

$(7,5) \notin S^3, \in S^2 \Rightarrow x_3 = 0$

$(7,5) \in S^2, \notin S^1 \Rightarrow x_2 = 1$

$(3,2) \in S^1 \notin S^0 \Rightarrow x_1 = 1$

$(5,4) \notin S^4 \Rightarrow x_5 = 0$

$(5,4)$

All pairs shortest path problem:

Let G= (V, E) be a directed graph consisting of n vertices and each edge is associated with a weight. The problem of finding the shortest path between all pairs of vertices in a graph is called all pairs shortest path problem.

→ This problem can be solved by using dynamic programming technique. The all pairs shortest path problem is to determine a matrix A such that A(i,j) is the length of a shortest path from vertex i to vertex j. Assume that this contains no cycles. If k is an intermediate vertex on this path. then the subpaths from i to k and from k to j are the shortest paths from i to k and k to j respectively. otherwise the path i to j is not shortest path.

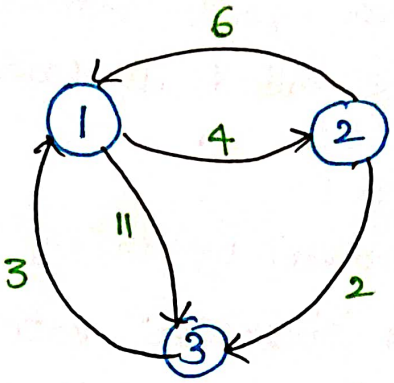
→ If k is intermediate vertex with highest index, then the path ~~is~~ i to k is the shortest path going through no vertex with index greater than k-1, similarly the path k to j is shortest path going through no vertex with index greater than k-1

→ The shortest path can be computed using following recursive method

$A^k(i,j) = w(i,j)$, if $k=0$

$= \min\{A^{k-1}(i,j), A^{k-1}(i,k) + A^{k-1}(k,j)\}$, if $k \geq 1$.

Example:



Graph (G)

Cost adjacency matrix, $A^0(i,j) = w(i,j) =$

	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

Step 1: For $k=1$, i.e. going from i to j through the intermediate vertex '1'.

$i=1$,

$A^1(1,1) = \min\{A^0(1,1), A^0(1,1) + A^0(1,1)\} = 0$

$A^1(1,2) = \min\{A^0(1,2), A^0(1,1) + A^0(1,2)\} = \min\{4, 0+4\} = 4$

$A^1(1,3) = \min\{A^0(1,3), A^0(1,1) + A^0(1,3)\} = \min\{11, 0+11\} = 11$

$i=2$.

$A^1(2,1) = \min\{A^0(2,1), A^0(2,1) + A^0(1,1)\} = \min\{6, 6+0\} = 6$

$A^1(2,2) = \min\{A^0(2,2), A^0(2,1) + A^0(1,2)\} = \min\{0, 6+4\} = 0$

$A^1(2,3) = \min\{A^0(2,3), A^0(2,1) + A^0(1,3)\} = \min\{2, 6+11\} = 2$

i=3

$A^1(3,1) = \min\{A^0(3,1), A^0(3,1) + A^0(1,1)\} = \min\{3, 3+0\} = 3$

$A^1(3,2) = \min\{A^0(3,2), A^0(3,1) + A^0(1,2)\} = \min\{\infty, 3+4\} = 7$

$A^1(3,3) = \min\{A^0(3,3), A^0(3,1) + A^0(1,3)\} = \min\{0, 3+11\} = 0$

$$\therefore A^1(i,j) = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix} \end{array}$$

Step 2: For k=2, i.e, going from i to j through vertex '2'.

$A^2(1,1) = \min\{A^1(1,1), A^1(1,2) + A^1(2,1)\} = \min\{0, 4+6\} = 0$

$A^2(1,2) = \min\{A^1(1,2), A^1(1,2) + A^1(2,2)\} = \min\{4, 4+0\} = 4$

$A^2(1,3) = \min\{A^1(1,3), A^1(1,2) + A^1(2,3)\} = \min\{11, 4+2\} = 6$

$A^2(2,1) = \min\{A^1(2,1), A^1(2,2) + A^1(2,1)\} = \min\{6, 0+6\} = 6$

$A^2(2,2) = \min\{A^1(2,2), A^1(2,2) + A^1(2,2)\} = \min\{0, 0+0\} = 0$

$A^2(2,3) = \min\{A^1(2,3), A^1(2,2) + A^1(2,3)\} = \min\{2, 0+2\} = 2$

$A^2(3,1) = 3$

$A^2(3,2) = 7$

$A^2(3,3) = 0$

$$\therefore A^2(i,j) = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix} \end{array}$$

Step 3: For k=3, i.e, going from i to j through vertex '3'.

$A^3(1,1) = 0$, $A^3(1,2) = 4$, $A^3(1,3) = 6$

$A^3(2,1) = 5$, $A^3(2,2) = 0$, $A^3(2,3) = 2$

$A^3(3,1) = 3$, $A^3(3,2) = 7$, $A^3(3,3) = 0$

$$\therefore A^3(i,j) = \begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

This matrix gives the all pairs shortest path solution.

Time complexity of the Algorithm:

```
Algorithm. All pairs shortest path(W, A, n)
// W is weighted array matrix, n is the no.of vertices
// A is the cost of shortest path from vertex i to j.
{
    for i ← 1 to n do
    {
        for j ← 1 to n do
        {
            A[i,j] ← W(i,j)
        }
    }
    for k ← 1 to n do
    {
        for i ← 1 to n do
        {
            for j ← 1 to n do
            {
                A[i,j] ← min(A[i,j], A[i,k] + A[k,j]);
            }
        }
    }
}
```

∴ Time complexity for this method is $O(n^3)$.

Multistage Graph.

→ A multistage graph G=(V,E) is a directed graph in which vertices are partitioned into k>=2 disjoint set (set v_i) where [1<=i<=k]. In addition, if (u,v) is an edge E then u∈ v_i, v∈ v_{i+1}.

→ Let c(i,j) be the cost of edge (i,j). The cost of a path from (s to T) is the sum of costs of the edges on the path

→ The multistage graph problem is to find the minimum cost path from "s" to "T"

→ The value on edges are called cost of edges.

$$cost(i,j) = \min_{l \in V_{i+1}} \{c(j,l) + cost(i+1,l)\}$$

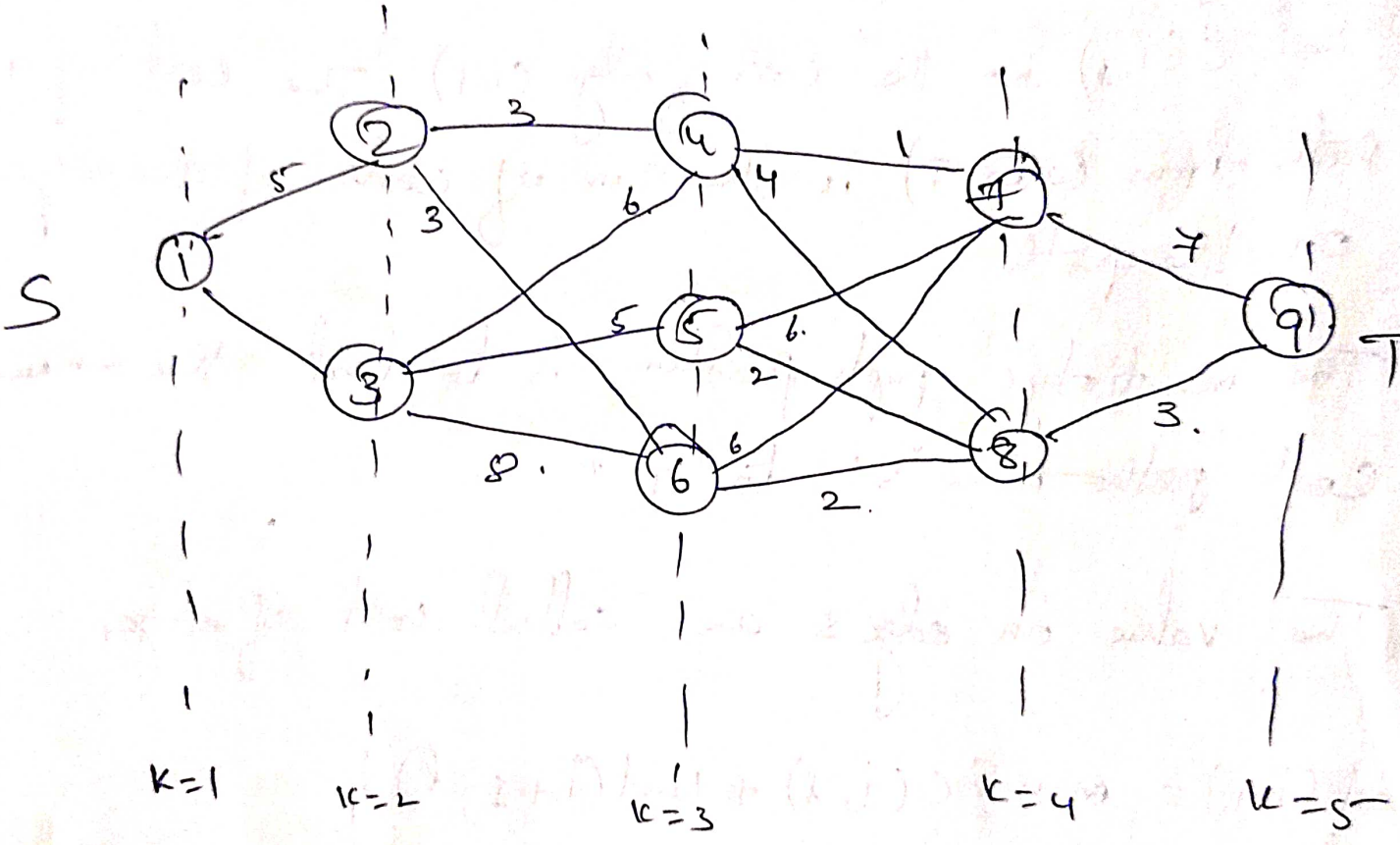
i.e cost(i,j) = cost(level, no. of nodes at that level)

'c' is a weight assigned on each edge.

'k' is the no. of stages.

Example 1:-

Consider the following example to understand the concept of multistage graph. In which we have to calculate the min. cost from path 's' to 't' and we have the value of k=5.



Solution.

According to formula we have to calculate the cost (i, j)

Step 1:-

1. Three nodes are selected as 'j' because at stage k-2=3, there are three nodes i.e 4, 5 & 6. so that we have the value of i=3 and for j=4, 5 & 6.

2.) We have three options here to choose minimum.

cost at stage $k = 3$

cost (3,4)

$cost(3,4) = min\{c(4,7) + cost(7,9)\}$ or $\{c(4,8) + cost(8,9)\}$

$cost(3,4) = min\{1+7\}$ or $\{4+3\}$.

$cost(3,4) = min\{8\}$ or $\{7\}$. →

$cost(3,4) = 7$

cost (3,5)

$cost(3,5) = min\{c(5,7) + cost(7,9)\}$ or $\{c(5,8) + cost(8,9)\}$

$cost(3,5) = min\{6+7\}$ or $\{2+3\}$

$cost(3,5) = min\{13\}$ or $\{5\}$

$cost(3,5) = 5$

cost (3,6)

$cost(3,6) = min\{c(6,7) + cost(7,9)\}$ or $\{c(6,8) + cost(8,9)\}$

$cost(3,6) = min\{6+7\}$ or $\{2+3\}$

$cost(3,6) = min\{13\}$ or $\{5\}$

$cost(3,6) = 5$

Note :- Here in step 1 we can select the node 8. because it gives us the minimum path from k-2 to k

Step 2:-

Cost (k-3, j)

1.) Two nodes are selected. as 'j' because at stage k-3 = 2, there are two nodes i.e 2 & 3. So we have the value of i = 2 and for j = 2 & 3

2.) we have three options here to choose minimum cost at stage k = 2.

Cost (2, 2)

Cost(2,2) = min{ c(2,4) + cost(4,8) + cost(8,9) } or { c(2,6) + cost(6,8) + cost(8,9) }

Cost(2,2) = min{3 + 4 + 3} or {3 + 2 + 3}

cost(2,2) = min{10} or {8}.

cost(2,2) = 8.

Cost(2,3)

Cost (2,3) = min{c(3,4) + cost(4,8) + cost(8,9)} or
{ c(3,5) + cost(5,8) + cost(8,9) } or
{ c(3,6) + cost(6,8) + cost(8,9)}.

Cost(2,3) = min{6 + 4 + 3} or { 5 + 2 + 3 }. or { 8 + 2 + 3 }

Mul

cost(2,3) = min{13} or {10} or {13}.

cost(2,3) = 10

Note :- node 6 is selected as it has minimum cost.

Steps :-

cost(k-4, j)

1.) only one node is selected as 'j' because at stage k-4=1, so there are only one node i.e 1 and we have the value of i=1 and for j=1

2.) we have three options here to choose min cost at stage k=1

cost(1,1)

cost(1,1) = min{c(1,2) + cost(2,6) + cost(6,8) + cost(8,9)} or
{c(1,3) + cost(3,6) + cost(6,8) + cost(8,9)}.

cost(1,1) = min{5+3+2+3} or {2+8+2+3}.

cost(1,1) = min{13} or {15}.

cost(1,1) = min{13} or {15}.

cost(1,1) = 13.

Note :- node 2 is selected as it has min. cost.

∴ Decision table:-

$1 \rightarrow 2 \rightarrow 6 \rightarrow 8 \rightarrow 9$ is having the minimum cost from 's' to 't'.

Multistage Graph using Dynamic Programming

- A multistage graph $G=(V,E)$ is a directed graph in which vertices are partitioned into $k >= 2$ disjoint set $(set\ v_i)$ where $[1<=i<=k]$. In addition, if (u,v) is an edge $\in E$ then $u \in v_i$, $v \in v_{i+1}$
- Let $c(i,j)$ be the cost of edge(i,j). The cost of path from $(s$ to $T)$ is the sum of costs of the edges on the path.
- The multistage graph problem is to find the minimum cost path from "S" to "T"
- The value on edges are called cost of edges.

$$cost(i,j) = \min \{ c(j,l) + cost(i+1, l) \}$$

$l \in v_{i+1}$

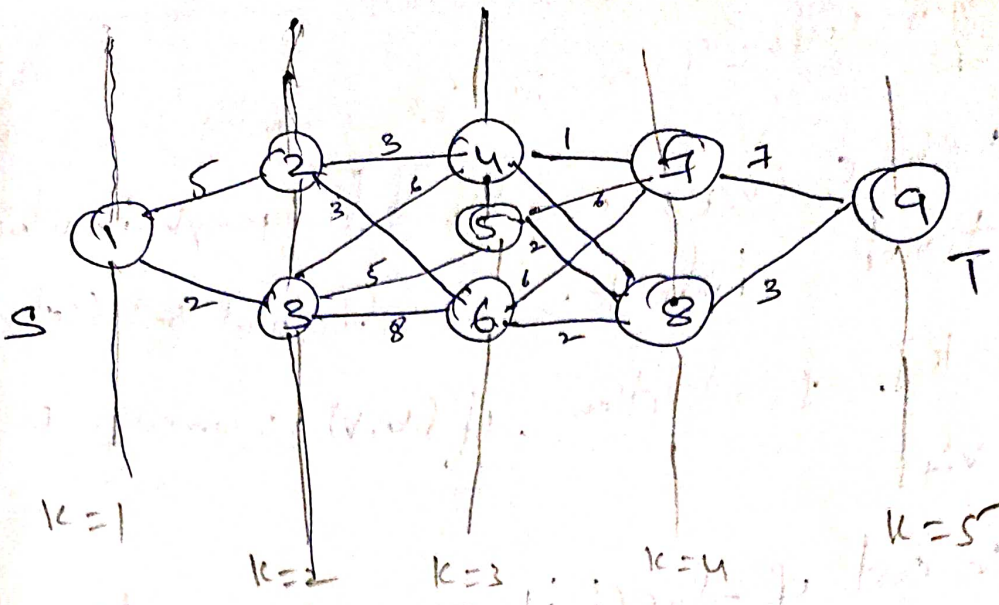
i.e cost(i, j) = cost (level, no. of nodes at that level)

'c' is a weight assigned on each edge.

'k' is the no. of stages.

Examples:-

Consider the following example to understand the concept of multistage graph. In which we have to calculate the min cost from path 'S' to 'T' and we have the value of k=5



Solution

According to formula we have to calculate the cost(i,j)

Step 1:-

cost(K-2, j)

1. Three nodes are selected as 'j' because at stage K-2 = 3, there are three nodes i.e 4, 5 & 6. So that we have the value of i = 3 and for j = 4, 5 & 6.

2.) We have three options have to choose minimum cost at stage k = 3.

cost(3,4) $l \in V_{i+1}$.

cost(3,4) = min{c(4,7) + cost(7,9)} or {c(4,8) + cost

cost(3,4) = min{1+7} or {4+3}

= min{8} or {7}.

cost(3,4) = 7

cost(3,5)

cost(3,5) = min{c(5,7) + cost(7,9)} or {c(5,8) + cost(8,9)}

= min{6+7} or {2+3}

= min{13} or {5}

cost(3,5) = 5

Cost(3,6) :-

Cost(3,6) = Min{c(6,7) + cost(7,9)} or {c(6,8) + cost(8,9)}

= min{6+7} or {2+3}

= min{13} or {5}.

cost(3,6) = 5

note:- Here in step 1 we can select the node 8 because it gives us the minimum path from k-2 to k.

Step 2:- cost(k-3, j)

1.) Two nodes are selected as "j" because at stage k-3=2 there are two nodes i.e 2 & 3 so we have the value of i = 2 and for j = 2 & 3

2.) we have three options here to choose minimum cost at stage k=2

cost(2,2)

cost(2,2) = min{c(2,4) + cost(4,8) + cost(8,9)} or {c(2,6) + cost(6,8) + cost(8,9)}

= min{3+4+1} or {3+2+3}

= min{10} or {8}

cost(2,2) = 8

cost(2,3)

cost(2,3) = min{c(3,4) + cost(4,8) + cost(8,9)} or {c(3,5) + cost(5,8) + cost(8,9)} or

{c(3,6) + cost(6,8) + cost(8,9)}

= min{6+4+3} or {5+2+3} or {8+2+3}

= min{13} or {10} or {13}

cost(2,3) = 10.

note:- node 6 is selected as it has minimum cost.

Step 3:- cost(k-4, j)

1. only one node is selected as "j" because at stage k-4=1, so there are only one node i.e 1 and we have the value of i=1 and for j=1

2.) we have three options here to choose minimum cost at stage k=1

cost (1,1).

$$cost(1,1) = \min\{(1,2) + cost(2,6) + cost(6,8) + cost(8,9)\} \text{ or } \{(1,3) + cost(3,6) + cost(6,8) + cost(8,9)\}.$$

$$cost(1,1) = \min\{5+3+2+3\} \text{ or } \{2+8+2+3\}.$$

$$cost(1,1) = \min\{13\} \text{ or } \{15\}$$

$$= 13$$

Note:- node 2 is selected as it has minimum cost.

Decision table.

1 → 2 → 6 → 8 → 9 is having the min. cost from "s" to "T".